

Contents

guide Mamba: Linear-Time Sequence Modeling with Selective State Spaces	1
0. 这篇 guide 的读法	1
1. 论文地位和当时语境	2
2. 原论文结构地图	3
3. 必须掌握的概念	3
4. 从 S4 到 Mamba 的核心数学	4
5. 为什么 LTI SSM 会在离散语言上吃亏	4
6. S6: 选择性状态空间模型	5
7. Delta 为什么像 RNN gate	6
8. 张量级别图示	6
9. 为什么 selective scan 是硬件问题	7
10. scan 为什么可并行	8
11. Mamba block 架构	9
12. 代码样例: 选择性记忆	9
13. 代码样例: S6 reference recurrence	10
14. 实验证据链	10
15. 论文没有证明什么	12
16. 对现在 LLM 学习的意义	12
17. 和本仓库的连接	13
18. 一个 30-60 分钟本地实验	13
19. 用 AI agent 学这篇论文的方法	14
20. 闭卷掌握检查	14
21. 一句话总结	14

guide_Mamba: Linear-Time Sequence Modeling with Selective State Spaces

原论文: Mamba: Linear-Time Sequence Modeling with Selective State Spaces

本地原文 PDF: [learning/ssm-hybrid/paper/01_mamba.pdf](#)

作者: Albert Gu, Tri Dao

arXiv 版本: 2023 初版, 2024-05-31 v2

本地代码: [learning/ssm-hybrid/src/](#)

0. 这篇 guide 的读法

这篇导读的目标是让你读完以后，能基本复述 Mamba 论文的 story、公式、架构、实验和局限。原文 PDF 仍然值得打开，因为 Figure 1, Figure 2, Figure 3 和各个实验图表有很多视觉信息；但如果你先把这份 guide 读透，再读原文会顺很多。

读 Mamba 不要把它当成一句口号：“用 SSM 替代 Transformer”。更准确的理解是：

- Transformer 的 attention 不压缩上下文，所以表达力强，但 KV cache 和二次训练成本很贵。
- 传统 SSM/RNN 压缩上下文，所以推理便宜，但固定状态必须决定该记什么，容易丢掉内容相关信息。
- Mamba 的核心问题是：能不能在保持 recurrent 线性复杂度的同时，让状态更新具备内容选择能力。

- 作者的答案是 selective SSM: 让 Delta, B, C 由当前输入 token 决定。
- 这样会失去传统 SSM 的 convolution 快路, 所以作者又设计 hardware-aware selective scan, 把 recurrence 在 GPU 上做得足够快。

所以这篇论文不是单点技巧, 而是一个连环交换:

Transformer attention:

```
keeps all previous tokens
-> strong content routing
-> expensive KV cache and quadratic training attention
```

LTI SSM:

```
compresses context into fixed state
-> cheap recurrent inference and convolutional training
-> weak content-based selection
```

Mamba selective SSM:

```
compresses context but lets token control write/read/forget
-> strong content selection
-> needs hardware-aware scan to regain efficiency
```

1. 论文地位和当时语境

Mamba 发表时, 长上下文模型路线大致有几类:

- 继续优化 Transformer attention, 例如 FlashAttention、稀疏 attention、滑窗 attention、线性 attention。
- 用 long convolution 或 gated convolution 做 subquadratic sequence model, 例如 Hyena。
- 用 structured state space model, 也就是 S4 一类模型, 利用状态空间和卷积形式做长序列。
- 用 recurrent 模型重新回到常数 KV cache 的推理模式, 例如 RWKV, RetNet 等路线。

这些路线各有吸引力, 但有一个共同尴尬: 很多 attention-free 模型在长序列、音频、连续信号上有亮点, 却很难在 language modeling 上达到强 Transformer 的质量。

语言和代码这类离散、高信息密度数据对模型的 content-based reasoning 要求更高。不是只要记住“第 1000 步之前的信息”就够了, 模型还要知道哪一个 token 值得写入状态, 哪一段历史应该忽略, 当前 token 是否触发某个旧上下文。

Mamba 的贡献在这里:

- 它把传统 SSM 的问题重新表述为“缺少 selection mechanism”。
- 它用很简单的方式加入 selection: 让 Delta, B, C 依赖输入 x_t 。
- 它指出这会破坏 LTI SSM 的 convolution 训练路径, 并用 scan + kernel fusion + recomputation 解决工程瓶颈。
- 它把 SSM 和 gated MLP 合成一个 homogenous block, 不再交替使用 attention block 和 MLP block。
- 它在语言、DNA、音频、效率和合成任务上建立证据链, 说明 selective SSM 不是只在一个 toy task 上好看。

这就是为什么 Mamba 重要: 它不是第一篇 SSM, 也不是第一篇 recurrent LLM, 但它把“内容选择 + 线性复杂度 + GPU 友好实现 + 统一 block”同时放到了一个可训练、可扩展的架构里。

2. 原论文结构地图

你读原文时建议按下面顺序慢读:

- Abstract 和 Introduction: 先抓住三件事, content-based reasoning、hardware-aware scan、no attention or MLP blocks。
- Section 2 State Space Models: 读 SSM 的 continuous form、discrete recurrence、convolution form、LTI 概念。
- Figure 1: 这是整篇论文最重要的系统图。注意大状态 h 的形状是 $B \times L \times D \times N$, 不能随便 materialize 到 HBM。
- Section 3.1 和 Figure 2: 读 selective copying 和 induction heads。这里不是小玩具, 而是作者给 selection 的动机。
- Section 3.2 和 Algorithms 1/2: 对比 S4 和 S6, 重点看 Delta, B, C 的 shape 从固定参数变成带长度维度的输入函数。
- Section 3.3: 读 selective scan 的工程理由。核心不是“scan”这个词, 而是 HBM/SRAM 之间的数据搬运。
- Section 3.4 和 Figure 3: 读 Mamba block 如何把 H3 block 和 gated MLP 合在一起。
- Section 3.5: 读 Delta 和 RNN gate 的关系。这是理解 Mamba 直觉的钥匙。
- Section 4: 读证据链。先看 synthetic, 再看 language, 再看 DNA/audio, 再看 efficiency 和 ablation。
- Section 5 Discussion: 认真读局限, 尤其是 continuous-discrete spectrum 和 scaling 说明。

3. 必须掌握的概念

SSM, state space model

SSM 把序列处理成一个隐藏状态 h 。每来一个输入 x_t , 状态更新一次, 再从状态读出 y_t 。你可以把它想成一个受控动态系统:

```
old state  $h_{t-1}$ 
current input  $x_t$ 
      |
      v
state update rule
      |
      v
new state  $h_t$ 
      |
      v
output  $y_t$ 
```

Structured SSM

普通状态空间模型太泛。深度学习里的 S4/structured SSM 会给 A 矩阵加结构, 例如 diagonal 或低秩结构, 让它能高效计算, 并有长程记忆的 inductive bias。

LTI, linear time invariance

LTI 指动态规则在所有时间步都一样。也就是 Delta, A, B, C 不随着 t 或 x_t 变化。这给传统 SSM 带来巨大好处: recurrence 可以展开成 convolution kernel, 从而并行训练。但代价是模型不能根据当前 token 内容改变写入和遗忘方式。

Selection mechanism

selection 是 Mamba 的中心概念。它不是普通 gating 的同义词，而是沿 sequence dimension 控制信息传播。它要回答：

- 当前 token 是否应该写进状态。
- 当前状态是否应该保留。
- 当前状态是否应该影响输出。
- 遇到边界或噪声时是否应该 reset 或 ignore。

Selective scan

当参数随输入变化后，传统 convolution kernel 不存在了，只能回到 recurrence。

但 recurrence 的每一步可以写成 affine map，affine map 的 composition 是 associative 的，所以可以用 parallel scan 思路并行化。

Mamba 的工程贡献是把这个 scan 做成 GPU 友好的 fused kernel。

Hardware-aware

这里的 hardware-aware 不是泛泛地说“GPU 加速”。它具体指：

- 不把 shape 为 $B \times L \times D \times N$ 的大状态完整写到 GPU HBM。
- 在 SRAM 里完成 discretization 和 recurrence 的局部计算。
- forward 不保存所有中间状态，backward 需要时 recompute。
- 用 kernel fusion 减少 HBM 读写。

4. 从 S4 到 Mamba 的核心数学

原文 Section 2 从连续时间系统写起：

$$\begin{aligned}h'(t) &= A h(t) + B x(t) \\ y(t) &= C h(t)\end{aligned}$$

直觉是：h 是系统状态，x 是外部输入，A 决定状态自己怎么演化，B 决定输入怎么写入状态，C 决定状态怎么读成输出。

离散化后，得到序列 recurrence：

$$\begin{aligned}h_t &= A_{\text{bar}} h_{t-1} + B_{\text{bar}} x_t \\ y_t &= C h_t\end{aligned}$$

如果 A_{bar} , B_{bar} , C 对所有 t 都一样，这就是 LTI。它可以展开成 convolution：

$$\begin{aligned}K &= (C B, C A B, C A^2 B, \dots) \\ y &= x * K\end{aligned}$$

这正是 S4 一类模型高效训练的基础：训练时整段序列可见，用 convolution 并行算；推理时一个 token 一个 token 到来，用 recurrence 算。

但 Mamba 论文的关键判断是：这个 LTI 假设太强了。语言不是均匀采样的平滑信号。一个 token 是否重要取决于它的内容和上下文，而不是只取决于它的位置。

5. 为什么 LTI SSM 会在离散语言上吃亏

论文用 Figure 2 讲了两个 synthetic tasks。

Copying task

普通 copying 的相关 token 和输出之间间隔固定。LTI convolution 只要学到一个固定延迟 kernel，就能解决。这个任务更像时间对齐，不一定需要理解内容。

Selective copying task

选择性复制把相关 token 放在随机位置，中间夹很多无关 token。模型必须看 token 内容来决定：

- colored token: 写入状态。
- white/noise token: 忽略。
- 输出阶段: 从状态读出之前写入的内容。

固定 convolution kernel 很难做这件事，因为输入到输出的距离在变。

Induction heads task

induction heads 来自 mechanistic interpretability。简化说，如果序列里之前出现过“Harry Potter”，那么第二次看到“Harry”时，模型应该预测“Potter”。

这要求模型做 associative recall，而不是只记一个固定位置。

作者想证明的不是“Mamba 会做玩具任务”。他们想用 toy task 指出一个结构性问题：

LTI SSM has fixed dynamics:

- same update rule at every step
- > cannot write/ignore based on token content
- > struggles with variable spacing and associative recall

Selective SSM has input-dependent dynamics:

- update rule changes with x_t
- > can decide what enters state
- > can preserve relevant state through long noise spans

6. S6: 选择状态空间模型

原文把传统 SSM/S4 和 selective SSM/S6 放在 Algorithms 1/2 中对比。

传统 S4 的形状可以简化理解为：

```
Input x:      (B, L, D)
A:            (D, N)
B:            (D, N)
C:            (D, N)
Delta:        (D)
Output y:     (B, L, D)
```

这里 B, C, Delta 没有长度维度 L。它们对所有位置一样。

S6 的变化是：

```
Input x:      (B, L, D)
A:            (D, N), still a learned structured parameter
B_t:          (B, L, N), produced from x
C_t:          (B, L, N), produced from x
Delta_t:      (B, L, D), produced from x
A_bar, B_bar: (B, L, D, N), after discretization
Output y:     (B, L, D)
```

作者具体采用:

```
B_t      = Linear_N(x_t)
C_t      = Linear_N(x_t)
Delta_t = softplus(parameter + Broadcast_D(Linear_1(x_t)))
```

这里三个细节很重要:

- A 保持为结构化参数, 没有直接让 A 也随输入变化, 主要为了简化和稳定。
- Delta 通过 softplus 保证为正, 并和离散化/门控直觉相连。
- B 和 C 变成输入函数, 分别控制写入状态和从状态读出。

最小的 S6 recurrence 可以写成:

```
for t in range(L):
    A_bar_t, B_bar_t = discretize(Delta_t, A, B_t)
    h_t = A_bar_t * h_{t-1} + B_bar_t * x_t
    y_t = C_t * h_t
```

注意这里的 * 是逐元素或按结构化矩阵的乘法, 不是完整 dense 矩阵乘法。真实实现按 batch、channel、state dimension 展开。

7. Delta 为什么像 RNN gate

Section 3.5 的 Theorem 1 很适合建立直觉。当 state dimension $N = 1$, $A = -1$, $B = 1$, 并且 Delta 由输入通过 softplus 产生时, selective SSM 可以写成类似 gate 的形式:

```
g_t = sigmoid(Linear(x_t))
h_t = (1 - g_t) h_{t-1} + g_t x_t
```

这说明 Delta 不只是一个步长超参, 它可以控制当前输入和旧状态的权衡:

- g_t 接近 1: 写入当前输入, 旧状态被覆盖。
- g_t 接近 0: 忽略当前输入, 保留旧状态。
- 介于中间: 做平滑混合。

用 selective copying 来看:

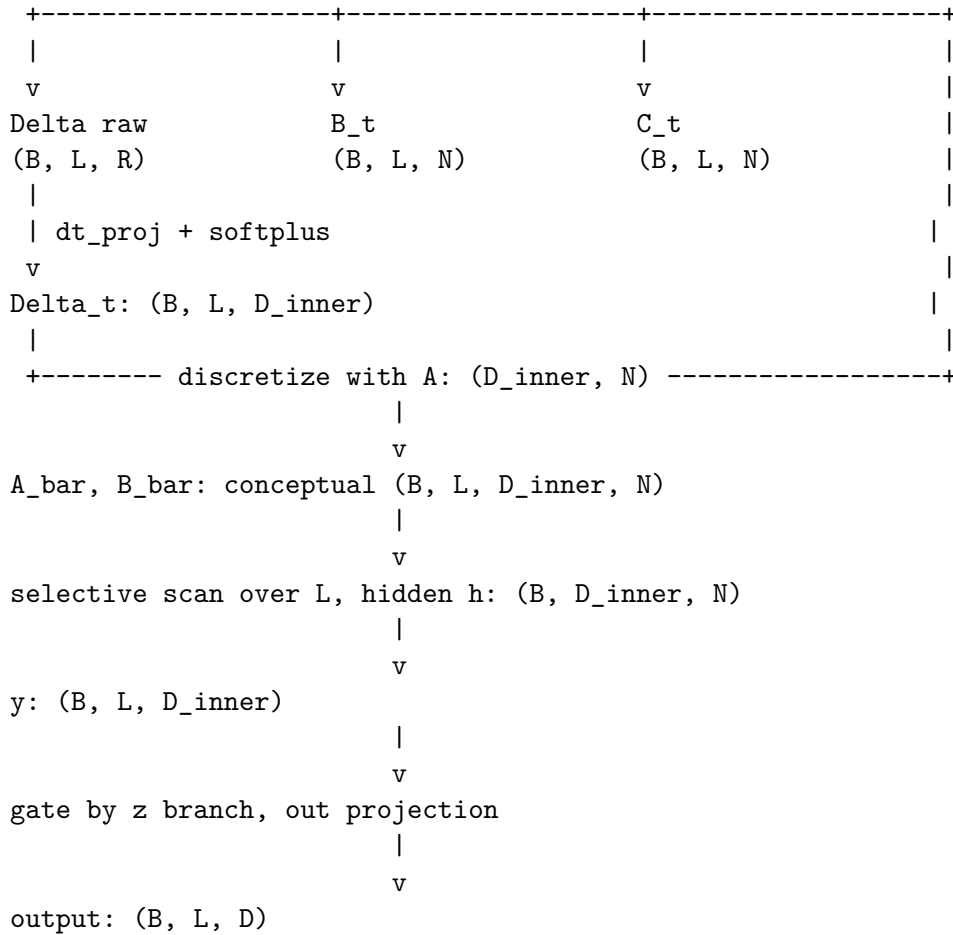
```
values: [5, 0, 0, 9, 0, 0]
gate:   [1, 0, 0, 1, 0, 0]
state:  [5, 5, 5, 9, 9, 9]
```

这就是内容相关压缩。状态仍然很小, 但它不是盲目平均所有历史, 而是学习把相关 token 写进去, 把无关 token 过滤掉。

8. 张量级别图示

下面这张图把 S6 的张量流画成 PDF 友好的版本:

```
x: (B, L, D)
|
| in projection and local causal conv
v
u: (B, L, D_inner)
|
| x_proj
```



概念上会出现 $B \times L \times D_{\text{inner}} \times N$ 的大张量，但高效实现不能把它完整写到 HBM。这正是 Section 3.3 的问题。

9. 为什么 selective scan 是硬件问题

传统 LTI SSM 的训练快路是 convolution。但一旦 Δ_t, B_t, C_t 随 x_t 变化，kernel K 不再固定：

LTI:

same A, B, C at all positions
 -> one convolution kernel K can represent all positions

Selective:

$A_{\text{bar}_t}, B_{\text{bar}_t}, C_t$ depend on x_t
 -> no single fixed convolution kernel
 -> must compute recurrence

天真的 recurrence 有两个麻烦:

- 顺序依赖: h_t 依赖 h_{t-1} 。
- 大状态: 每个 batch、position、channel 都有 N 维状态，形状是 $B \times L \times D \times N$ 。

Mamba 的观察是:

- recurrent mode 的 FLOPs 是 $O(B * L * D * N)$ 。
- convolutional SSM 往往是 $O(B * L * D * \log L)$ 加上 kernel 生成成本。
- 当 N 不大时，recurrence 的 FLOPs 不一定比 convolution 更差。

- 真正危险的是内存 IO，尤其是把大中间状态写进 HBM。

所以硬件算法的目标是:

Do not:

materialize A_{bar} , B_{bar} , C_{t} , h_{t} for every position in HBM

Do:

load compact parameters from HBM

compute discretization in SRAM

scan/recur in SRAM

write only final y back to HBM

recompute hidden states during backward when needed

这和 FlashAttention 的精神很像: 不是改变数学结果, 而是改变数据搬运路径。

10. scan 为什么可并行

对单个 state 维度, recurrence 可以写成 affine map:

$$h_t = a_t h_{t-1} + b_t$$

每一步都是一个函数:

$$f_t(h) = a_t h + b_t$$

两个函数复合:

$$f_2(f_1(h))$$

$$= a_2 (a_1 h + b_1) + b_2$$

$$= (a_2 a_1) h + (b_2 + a_2 b_1)$$

所以每一步可以表示成 pair:

$$(a, b)$$

pair 的 composition:

$$(a_2, b_2) \text{ after } (a_1, b_1)$$

$$= (a_2 * a_1, b_2 + a_2 * b_1)$$

这个操作是 associative 的, 因此可以做 parallel prefix scan。

真实 Mamba kernel 比这个复杂得多, 因为它要处理 batch、channel、state dimension、discretization、backward recomputation 和 GPU memory hierarchy。但学习上先记住这点就够了:

recurrence is sequential in appearance

affine composition is associative

scan exposes parallelism

hardware-aware fusion makes it fast enough

本仓库新增了一个最小代码文件:

learning/ssm-hybrid/src/mamba_original_minimal.py

其中 sequential_affine_prefix_scan 就是在教学层面复现这个想法。

11. Mamba block 架构

原文 Figure 3 把 H3 block 和 gated MLP 合成一个 Mamba block。它的高层流程是:

```
input x: (B, L, D)
|
| Linear projection to 2 branches
+-----+
|                                     |
|                                     |
v                                     v
x branch                             z branch
(B, L, D_inner)                     (B, L, D_inner)
|                                     |
| causal depthwise conv              |
| SiLU                               |
v                                     v
parameter projection for Delta, B, C
|
| selective SSM scan
v
y: (B, L, D_inner)
|
| multiply by SiLU(z)
v
output projection
|
v
block output: (B, L, D)
```

和 Transformer block 对比:

- Transformer 通常是 MHA block + MLP block 交替。
- Mamba block 把 sequence mixing 和 gated MLP 风格的 channel mixing 合在一个 block 里。
- 论文实验里 expansion factor E 固定为 2, 并用两个 Mamba block 的参数量大致匹配一个 Transformer 的 MHA+MLP 组合。
- Mamba 不使用 attention, 也不使用单独的 MLP block。

和本仓库代码对应:

- mamba_block.py 里的 MambaBlock 是 naive PyTorch 教学实现。
- selective_scan_naive 用 for-loop 写出 recurrence, 方便检查公式。
- mini_mamba.py 把多个 MambaBlock 堆成一个小语言模型。
- s4_naive.py 是 LTI SSM 对照。
- mamba_original_minimal.py 是这次新增的论文机制最小解释。

12. 代码样例: 选择性记忆

下面这个片段就是 Theorem 1 的最小直觉版:

```
import torch

def gated_selective_memory(values, gate):
    h = torch.tensor(0.0)
```

```

out = []
for x_t, g_t in zip(values, gate):
    h = (1.0 - g_t) * h + g_t * x_t
    out.append(h)
return torch.stack(out)

values = torch.tensor([5.0, 0.0, 0.0, 9.0, 0.0, 0.0])
gate = torch.tensor([1.0, 0.0, 0.0, 1.0, 0.0, 0.0])

print(gated_selective_memory(values, gate))
# tensor([5., 5., 5., 9., 9., 9.])

```

这个例子虽然小，但它说明了论文最核心的内容: fixed-size state 并不必然等于信息损失严重，关键在于状态更新是否能根据输入内容选择。

13. 代码样例: S6 reference recurrence

本仓库的 `mamba_original_minimal.py` 还包含一个单通道 selective SSM:

```

def selective_ssm_reference(u, delta, A, B, C):
    A_bar, B_bar = discretize_zoh_diagonal(delta, A, B)
    h = torch.zeros_like(A)
    out = []
    for t in range(u.shape[0]):
        h = A_bar[t] * h + B_bar[t] * u[t]
        out.append(torch.dot(C[t], h))
    return torch.stack(out)

```

它和论文的差别:

- 只演示一个 channel，不处理完整 D_{inner} 。
- 用 CPU for-loop，不做 fused scan。
- 只服务理解，不服务性能。

但它保留了论文机制:

- A 是结构化参数。
- Delta, B, C 随位置变化。
- 先 discretize，再 recurrence。
- y_t 从当前 hidden state 读出。

14. 实验证据链

Mamba 的证据链不是只靠一个语言模型表格，而是多层组合。

第一层: synthetic tasks

Selective Copying 的 Table 1 说明:

- S4 无 gate 只有 18.3 accuracy。
- 把 inner layer 从 S4 换成 S6 后到 97.0。
- H3 架构 + S6 到 99.7。
- Mamba 架构 + S6 到 99.8。

这说明真正关键的是 selection mechanism，而不只是外层 gated architecture。

Induction Heads 的 Table 2 和相关图说明：

- 模型训练在 length 256。
- Mamba/selective SSM 可以外推到 length 1,048,576。
- 其他方法基本无法超过很短的外推范围。

这支持作者的动机：selective state 可以做某种内容相关记忆，而不是只做固定时间对齐。

第二层：language modeling

Section 4.2 在 The Pile 上做 autoregressive language modeling。重点不是“Mamba 打败所有 Transformer”，而是更细：

- baseline 包括 GPT3-style Transformer。
- 还包括更强的 Transformer++ recipe，例如 RoPE、SwiGLU、RMSNorm、no bias、更高学习率。
- 模型规模从 about 125M 到 about 1.3B 做 scaling law。
- Figure 4 的结论是：Mamba 是第一个 attention-free 模型，能够匹配强 Transformer++ recipe，尤其在长 context 下表现更好。

Table 3 的 zero-shot 评估也很重要：

- Mamba-130M 到 Mamba-2.8B 在同规模上通常优于 Pythia/RWKV 等开源模型。
- Mamba-2.8B 的平均 commonsense reasoning 指标达到 63.3。
- Pythia-2.8B 是 59.1，Pythia-6.9B 是 61.7，RWKV-7.4B 是 62.5。
- 这支持论文摘要里的说法：Mamba-3B 级别可以匹配更大 Transformer 的质量。

第三层：DNA

DNA 是离散序列，但需要超长依赖。Section 4.3 的设计很适合证明 Mamba 的长 context 价值：

- HG38 human genome 预训练。
- 先固定短 context length 1024，看模型 size scaling。
- 再固定模型大小，把 sequence length 从 1024 增加到 1,048,576。

Figure 5 的结论：

- Mamba 随模型大小增长更平滑。
- 在最大 about 40M 参数时，Mamba 可以用 about 3x-4x 更少参数匹配 Transformer++ 和 HyenaDNA。
- 当 context length 增加到 1M，Mamba perplexity 继续改善，而 HyenaDNA 变差。

这支持 Section 3.5 的直觉：selective model 可以过滤无关长上下文，而 LTI convolution 在很长窗口里可能聚合太多噪声。

第四层：audio

音频结果更微妙，也更值得读。Section 4.4 表明：

- 在 YouTubeMix 长音频预训练上，Mamba 优于 SaShiMi，并随更长 context 改善。
- 在 SC09 speech generation 上，小 Mamba 模型在 FID、IS、mIS、AM 等指标上优于多种 autoregressive、GAN、diffusion baseline。

但 Appendix 的 Figure 10 也指出：

- 对原始平滑音频波形，完全 selective 的 S6 不总是优于 LTI S4。
- 连续信号有时更适合 LTI 的 inductive bias。
- 这就是论文 Discussion 里的 continuous-discrete spectrum。

这个局限很重要: Mamba 不是说 selectivity 永远更好, 而是说离散、高信息密度数据需要内容选择。

第五层: efficiency

Figure 8 展示:

- core scan 在 sequence length 大于 2K 后快过 FlashAttention-2 的 core attention benchmark。
- optimized scan 比 PyTorch 标准 scan 快 20x-40x。
- Mamba 推理吞吐比同规模 Transformer 高 4x-5x。

原因是:

- Transformer autoregressive inference 需要 KV cache, batch 越大 cache 压力越大。
- Mamba 是 recurrent state, 单步推理不需要存全历史 KV。
- 更高 batch size 下吞吐优势明显。

第六层: ablation

Table 6 到 Table 10 是判断论文是否扎实的关键:

- Table 6: 换成 selective S6 后 perplexity 从约 10.x 降到 8.x, 验证 selection 的价值。
- Table 7: selective Delta 最重要, 但 Delta、B、C 一起 selective 效果最好。
- Table 8: 语言建模里 real-valued A 初始化比 complex S4D-Lin 更合适。
- Table 9: Delta projection 从 static 到 dim 1 就带来大收益, 继续增大维度收益较小。
- Table 10: 增大 state dimension N 只有在 B 和 C 也 selective 时才明显有用。

这条证据链非常漂亮: 它不是只说“Mamba 好”, 而是把“为什么选择性重要”和“哪些参数最重要”拆开证明。

15. 论文没有证明什么

读 Mamba 时要保持清醒。它很强, 但不是终局答案。

首先, 论文规模有限。作者自己在 Discussion 说, 实验主要低于当时最强开源 LLM 的大规模阈值。Mamba-3B 很有说服力, 但还不能证明任意大规模都优于 Transformer。

其次, Mamba 的 fixed state 仍然是压缩。attention 保留全部上下文, 理论上更容易做精确检索。Mamba 需要学会把必要信息压进状态, 遇到需要逐字精确回看长上下文的任务时, 未必天然占优。

第三, 速度优势依赖实现。天真的 Python loop Mamba 会很慢。论文贡献的一半在 hardware-aware fused scan。没有对应 kernel, 就不能期待论文里的 throughput。

第四, 连续信号上 selection 不总是好。音频 ablation 表明, 有些均匀采样、平滑的连续数据更适合 LTI inductive bias。

第五, Mamba 生态当时还没有 Transformer 那么成熟。fine-tuning、instruction tuning、RLHF、tool use、quantization、serving、debugging 等工程生态都需要后续验证。

16. 对现在 LLM 学习的意义

Mamba 的现代意义可以总结成四点:

- 它提醒我们, attention 的优势来自内容路由, 而不是“Transformer”这个名字本身。
- 它证明 fixed-state sequence model 不是只能做弱 RNN, 只要状态更新有 selection, 也能在语言上接近 Transformer。
- 它把模型架构和 kernel 设计绑在一起看, 说明算法论文不能脱离 GPU memory hierarchy。
- 它推动了后续 SSM、Mamba-2、hybrid attention-SSM、Jamba/Zamba 一类路线。

今天读 Mamba, 最值得学的不是“ 是否应该全盘替换 attention”, 而是三种能力:

- 建模能力: 如何让状态选择性压缩上下文。
- 数学能力: 如何从 recurrence、convolution、scan 三个视角看同一个模型。
- 工程能力: 如何把大中间状态留在 SRAM, 减少 HBM IO。

17. 和本仓库的连接

建议按这个顺序读代码:

1. learning/ssm-hybrid/src/common.py
 - 看 discretize_zoh 和 naive_scan。
 - 目标是把 SSM 公式变成代码。
2. learning/ssm-hybrid/src/s4_naive.py
 - 看 LTI SSM 的固定 A, B, C, Delta。
 - 目标是理解为什么它能 recurrent, 也能看作 convolution。
3. learning/ssm-hybrid/src/mamba_original_minimal.py
 - 看 gated_selective_memory、selective_ssm_reference、sequential_affine_prefix_scan。
 - 目标是对应论文 Section 3.2 和 3.3。
4. learning/ssm-hybrid/src/mamba_block.py
 - 看完整教学版 MambaBlock。
 - 目标是把 Delta/B/C projection、causal conv、z gate、out projection 串起来。
5. learning/ssm-hybrid/src/mini_mamba.py
 - 看如何堆 block 做语言模型。

本地测试:

```
.\.venv\Scripts\python.exe -m pytest learning\ssm-hybrid\src\tests -q
```

18. 一个 30-60 分钟本地实验

实验目标: 亲手验证 selection 与 fixed gate 的差别。

步骤:

1. 打开 learning/ssm-hybrid/src/mamba_original_minimal.py。
2. 修改 selective_copy_toy 里的 values, 把两个非零 token 放到更远的位置。
3. 让 colored_token_gate 只在非零 token 位置为 1。
4. 运行:

```
.\.venv\Scripts\python.exe learning\ssm-hybrid\src\mamba_original_minimal.py
```

你应该观察到:

- selective memory 会一直保留最近一次被 gate 选中的 token。
- fixed-gate memory 会被中间的 0 或噪声不断冲淡。
- 这就是 selective copying 的玩具版。

进阶实验:

1. 打开 tiny_s6_inputs。
2. 把 delta 的某些位置调大, 另一些位置调小。
3. 看 selective_ssm_reference 输出如何变化。
4. 用自己的话解释: 大 Delta 更像写入/重置, 小 Delta 更像保留/忽略。

19. 用 AI agent 学这篇论文的方法

你可以把 agent 当成论文陪练，而不是摘要机。

推荐流程：

1. 让 agent 先只解释 Figure 1，不许讲实验。
 - 目标是把 B, L, D, N 和 HBM/SRAM 讲清楚。
2. 让 agent 把 Algorithms 1/2 逐行对照。
 - 目标是看出 S4 到 S6 只是少数 shape 变化，但后果巨大。
3. 让 agent 根据 Theorem 1 写一个 10 行 toy example。
 - 目标是把 Delta/gate 直觉写进手指。
4. 让 agent 问你闭卷问题。
 - 例如“为什么 selection 会破坏 convolution 快路”。
 - 你必须先答，agent 再纠正。
5. 让 agent 把实验表格转成证据链。
 - 不要只问“结果如何”。
 - 要问“这个表支持哪个机制，不能支持哪个机制”。
6. 最后让 agent 改一个本地测试。
 - 例如让 fixed gate 反例更明显。
 - 这样知识会落到代码和调试里，而不是停在阅读印象里。

20. 闭卷掌握检查

读完后你应该能回答：

- 为什么 attention 的表达力强但推理 cache 贵。
- 为什么 LTI SSM 可以写成 convolution。
- 为什么 selective SSM 不能继续用固定 convolution kernel。
- Delta, B, C 分别控制什么。
- Theorem 1 如何把 Delta 和 RNN gate 联系起来。
- selective copying 为什么比普通 copying 更能暴露 LTI 缺陷。
- induction heads task 为什么和 in-context learning 有关系。
- Mamba block 中 causal conv、selective scan、z gate、out projection 各自做什么。
- hardware-aware selective scan 到底避免了什么 HBM 写入。
- 为什么 affine recurrence 可以用 scan 并行化。
- Table 7 为什么说 Delta 最重要，但 Delta/B/C 一起最好。
- 为什么音频 ablation 提醒我们 selection 不是万能。
- 本仓库里哪个文件对应 S4，对应 S6，对应 Mamba block，对应 toy scan。

21. 一句话总结

Mamba 的核心不是“RNN 回来了”，而是：用输入相关的 Delta/B/C 给 fixed-size state 加上内容选择能力，再用 GPU 友好的 selective scan 让这个 recurrence 在训练和推理上都可用。